

Automate Identification And Recognition Of Handwritten Text from An Image.

(Using Convolutional Recurrent Neural Network)

Name Of The Student : AMAREESHVAR R

Internship Project Topic : Automate identification and recognition of handwritten text from an image

Name of the Organization : HUMAN AI

Name of the Industry Mentor : SELF LEARNING

Name of the Institute : ST. JOSEPH'S INSTITUTE OF TECHNOLOGY- CHENNAI- TAMILNADU

Introduction

An optical character recognition problem is basically a type of image-based sequence recognition problem. And for sequence recognition problem, most suited neural networks are recurrent neural networks(RNN) while for an image-based problem most suited are convolution neural networks(CNN). To cop up with the OCR problems we need to combine both of these CNN and RNN.

We can break the implementation of CRNN network into following steps:

1. Setting Up kaggle.
2. Collecting Dataset
3. Preprocessing Data
4. Creating Network Architecture
5. Defining Loss Function
6. Training Model
7. Testing and Prediction
8. Plot Accuracy and Loss.
9. Get Best Model Index
10. Save the Model.

▼ Setting Up Kaggle in Google Colab.

This is optional method to run this model. This method is only for use of GPU on Google Colab. If one wants to use GPU on local machine then this step is not required. I used kaggle to load dataset in Google Colab. There are 4 steps to setting up kaggle in google colab.

1. Install Kaggle.
2. Create token.
3. Create Folder.
4. Get API link and download dataset.
5. Unzip the File.

Dataset

we used IAM handwritten dataset. This is good dataset total of 1.09 GB images. Here I have used only 7850 images for the training set and 876 images for validation dataset.

To download the dataset either you can directly download from Kaggle. For use download the data and unzip.

Installing Kaggle to use kaggle dataset on Google Colab.

```
!pip install kaggle
```

```
Requirement already satisfied: kaggle in /usr/local/lib/python3.6/dist-packages (1.5.13)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.6/dist-packages (2.8.1)
Requirement already satisfied: certifi in /usr/local/lib/python3.6/dist-packages (2020.12.5)
Requirement already satisfied: tqdm in /usr/local/lib/python3.6/dist-packages (4.61.2)
Requirement already satisfied: urllib3<1.25,>=1.21.1 in /usr/local/lib/python3.6/dist-packages (1.25.11)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.6/dist-packages (5.0.2)
Requirement already satisfied: requests in /usr/local/lib/python3.6/dist-packages (2.25.1)
Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.6/dist-packages (1.15.0)
Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.6/dist-packages (1.3)
Requirement already satisfied: idna<3,>=2.5 in /usr/local/lib/python3.6/dist-packages (2.10)
Requirement already satisfied: chardet<4,>=3.0.2 in /usr/local/lib/python3.6/dist-packages (3.0.4)
```

I uploaded IAM dataset of words on kaggle.

For more details of how to upload dataset on kaggle.

First grab your token from kaggle.

For more details of creating API on kaggle

Upload the json file got from kaggle.

```
from google.colab import files
files.upload() #upload kaggle.json
```



```
kaggle.json(application/json) - 67 bytes, last modified: 7/14/2020 - 100% done
Saving kaggle.json to kaggle (1).json
{'kaggle.json': b'{"username":"vijaydevane","key":"67147028d77bd467e124098f91cc7b2b"}
```

```
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/

!chmod 600 /root/.kaggle/kaggle.json
```

Copy the API link and paste with '!' to download the dataset.

```
!kaggle datasets download -d vijaydevane/iamdatasethtrwords
```

```
↳ iamdatasethtrwords.zip: Skipping, found more recently modified local copy (use --for
```

This code for unzip the file.

```
from zipfile import ZipFile
file_name = "iamdatasethtrwords.zip"
with ZipFile(file_name,'r') as zip:
    zip.extractall()
    print('Done')
```

```
↳ Done
```

▼ Importing necessary packages.

Installing Keras_tqdm.

```
!pip install keras_tqdm
```

```
↳ Requirement already satisfied: keras_tqdm in /usr/local/lib/python3.6/dist-packages
Requirement already satisfied: Keras in /usr/local/lib/python3.6/dist-packages (from
Requirement already satisfied: tqdm in /usr/local/lib/python3.6/dist-packages (from
Requirement already satisfied: scipy>=0.14 in /usr/local/lib/python3.6/dist-packages
Requirement already satisfied: h5py in /usr/local/lib/python3.6/dist-packages (from
Requirement already satisfied: numpy>=1.9.1 in /usr/local/lib/python3.6/dist-package
Requirement already satisfied: six>=1.9.0 in /usr/local/lib/python3.6/dist-packages
Requirement already satisfied: keras-applications>=1.0.6 in /usr/local/lib/python3.6
Requirement already satisfied: pyyaml in /usr/local/lib/python3.6/dist-packages (fro
Requirement already satisfied: keras-preprocessing>=1.0.5 in /usr/local/lib/python3.
```

```
import numpy as np
import cv2
import os
import pandas as pd
import string
import matplotlib.pyplot as plt
```

```
import os
from google.colab import drive #To use googel drive to get files.

from keras.preprocessing.sequence import pad_sequences

from keras.layers import Dense, LSTM, Reshape, BatchNormalization, Input, Conv2D, MaxPool2
from keras.models import Model
from keras.activations import relu, sigmoid, softmax
import keras.backend as K
from keras.utils import to_categorical
from keras.callbacks import ModelCheckpoint
from keras_tqdm import TQDMNotebookCallback

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler

import tensorflow as tf

#ignore warnings in the output
tf.compat.v1.logging.set_verbosity(tf.compat.v1.logging.ERROR)
```

▼ Tensorflow GPU

We used Google Colab GPU.

```
from tensorflow.python.client import device_lib

# Check all available devices if GPU is available
print(device_lib.list_local_devices())
sess = tf.compat.v1.Session(config=tf.compat.v1.ConfigProto(log_device_placement=True))
```



```

{
  "nbformat": 4,
  "nbformat_minor": 0,
  "metadata": {
    "colab": {
      "name":
        "useofpytesseract.ipynb",
      "provenance": [],
      "collapsed_sections": []
    },
    "kernel_spec": {
      "name": "python3",
      "display_name": "Python 3"
    }
  },
  "cells": [
    {
      "cell_type": "code",
      "metadata": {
        "id": "3Vh50vRoEfGJ",
        "colab_type": "code",
        "colab": {
          "base_uri":
            "https://localhost:8080/",
          "height": 357
        }
      },
      "outputId": "12b911f9-80eb-411a-fbb9-a5bc2fd84de6"
    },
    {
      "source": [
        "!pip install opencv-python\n",
        "!sudo apt-get install
        tesseract-ocr\n",
        "!sudo apt-get install
        libtesseract-dev\n",
        "!pip install pytesseract"
      ]
    }
  ]
}

```

```
],  
"execution_count": null,  
"outputs": [  
  {  
    "output_type": "stream",  
    "text": [  
      "Requirement already satisfied:  
      opencv-python in  
      /usr/local/lib/python3.6/dist  
      packages (4.1.2.30)\n",  
      "Requirement already satisfied:  
      numpy>=1.11.3 in  
      /usr/local/lib/python3.6/dist  
      packages (from opencv-python)  
      (1.18.5)\n",  
      "Reading package lists...  
      Done\n",  
      "Building dependency tree \n",  
      "Reading state information...  
      Done\n",  
      "tesseract-ocr is already the  
      newest version (4.00~git2288-  
      10f4998a-2).\n",  
      "The following package was  
      automatically installed and is  
      no longer required:\n",  
      " libnvidia-common-440\n",  
      "Use 'sudo apt autoremove' to  
      remove it.\n",  
      "\n0 upgraded, 0 newly installed,  
      0 to remove and 43 not  
      upgraded.\n",  
      "Reading package lists...  
      Done\n",  
      "Building dependency tree \n",  
      "Reading state information...  
      Done\n",  
      "libtesseract-dev is already  
      the newest version  
      (4.00~git2288-10f4998a-2).\n",  
      "The following package was  
      automatically installed and is  
      no longer required:\n",
```

```

" libnvidia-common-440\n",

"Use 'sudo apt autoremove' to
remove it.\n",

"0 upgraded, 0 newly installed,
0 to remove and 43 not
upgraded.\n",

"Requirement already satisfied:
pytesseract in
/usr/local/lib/python3.6/dist
packages (0.3.4)\n",

"Requirement already satisfied:
Pillow in
/usr/local/lib/python3.6/dist-
packages

(from pytesseract) (7.0.0)\n"
],

"name": "stdout"
}

],

},

{
"cell_type": "code",
"metadata": {
"id": "Cf86mhytEyNG",
"colab_type": "code",
"colab": {
"base_uri":
"https://localhost:8080/",
"height": 51
},
"outputId": "8ea05290-e922-
4a6e-82f3-e5c3cf08c2f8"
},
"source": [
"import cv2\n",

"from google.colab import
drive\n",

"import pytesseract\n",

"drive.mount('/content/gdrive')

```

```

\n",

"img = cv2.imread('gdrive/My
Drive/TcsInternship/spacingOutT
ext.gif')\n",

"print(\"Loaded\")"

],

"execution_count": null,

"outputs": [

{

"output_type": "stream",

"text": [

"Drive already mounted at
/content/gdrive; to attempt to
forcibly remount, call

drive.mount(\"/content/gdrive\"
, force_remount=True).\n",

"Loaded\n"

],

"name": "stdout"

}

],

},

{

"cell_type": "code",

"metadata": {

"id": "sthXEBfiFr12",

"colab_type": "code",

"colab": {

"base_uri":

"https://localhost:8080/",

"height": 170

},

"outputId": "a3a0ec49-892d-
4ffa-8d30-93dd5c437fba"

},

"source": [

"text =

pytesseract.image_to_string('gd

```



```
rive/My

Drive/TcsInternship/spacingOutT
ext.gif',lang='eng')\n",

"print(\"The Text is:\", text)"

],

"execution_count": null,


"outputs": [

{

"output_type": "stream",

"text": [

"The Text is: Let's get
some\n",

"indenting and _ word-
spacing\n",

"going. on.\n",

"\n",

"And how about some\n",

"letter-spacing, line-\n",

"height, and _ justified\n",

"\n",

"text-alignment?\n"]

}
```

```
with open('gdrive/My Drive/TcsInternship/HTR_Using_CRNN/Data/words.txt') as f:
    contents = f.readlines()
```

```
lines = [line.strip() for line in contents]
lines[0]
```

```
↳ Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive
'a01-000u-00-00 ok 154 408 768 27 51 AT A'
```

```
max_label_len = 0
```

```
char_list = "!\"'#$%&'()*+,-./0123456789:;?ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz
```

```
# string.ascii_letters + string.digits (Chars & Digits)
```

```
# or
```

```
# "!\"'#$%&'()*+,-./0123456789:;?ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz"
```

```
print(char_list, len(char_list))
```

```
def encode_to_labels(txt):
```

```
    # encoding each output word into digits
```

```
    dig_lst = []
```

```
    for index, chara in enumerate(txt):
```

```
        dig_lst.append(char_list.index(chara))
```

```
    return dig_lst
```

```
↳ !\"'#$%&'()*+,-./0123456789:;?ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopqrstuvwxyz 78
```

```
images = []
```

```
labels = []
```

```
RECORDS_COUNT = 10000
```

```
train_images = []
```

```
train_labels = []
```

```
train_input_length = []
```

```
train_label_length = []
```

```
train_original_text = []
```

```
valid_images = []
```

```
valid_labels = []
```

```
valid_input_length = []
```

```
valid_label_length = []
```

```
valid_original_text = []
```

```
inputs_length = []
```

```
labels_length = []
```

```
def process_image(img):
```

```
    """
```

```
    Converts image to shape (32, 128, 1) & normalize
```

```
    """
```



```

# processing on image
img = cv2.imread(filepath, cv2.IMREAD_GRAYSCALE)
try:
    img = process_image(img)
except:
    continue

# processing on label
try:
    label = encode_to_labels(word)
except:
    continue

if index % 10 == 0:
    valid_images.append(img)
    valid_labels.append(label)
    valid_input_length.append(31)
    valid_label_length.append(len(word))
    valid_original_text.append(word)
else:
    train_images.append(img)
    train_labels.append(label)
    train_input_length.append(31)
    train_label_length.append(len(word))
    train_original_text.append(word)

if len(word) > max_label_len:
    max_label_len = len(word)

if index >= RECORDS_COUNT:
    break

```

Generate Padded label (padded_label = pad_sequences(labels, maxlen=max_label_len, padding='post', value=len(char_list)))

```

train_padded_label = pad_sequences(train_labels,
                                   maxlen=max_label_len,
                                   padding='post',
                                   value=len(char_list))

```

```

valid_padded_label = pad_sequences(valid_labels,
                                   maxlen=max_label_len,
                                   padding='post',
                                   value=len(char_list))

```

```

train_padded_label.shape, valid_padded_label.shape

```

```

↳ ((7850, 16), (876, 16))

```

Convert to numpy array.

- `images = np.asarray(images)`
- `inputs_length = np.asarray(inputs_length)` •
`labels_length = np.asarray(labels_length)`

```
train_images = np.asarray(train_images)
train_input_length = np.asarray(train_input_length)
train_label_length = np.asarray(train_label_length)
```

```
valid_images = np.asarray(valid_images)
valid_input_length = np.asarray(valid_input_length)
valid_label_length = np.asarray(valid_label_length)
```

```
train_images.shape
```

```
↳ (7850, 32, 128, 1)
```

▼ Build Model (Network Architecture).

(Using Convolutional Recurrent Neural Network)

This network architecture is inspired by [this paper](#). Let's see the steps that we used to create the architecture:

Input shape for our architecture having an input image of height 32 and width 128. Here we used seven convolution layers of which 6 are having kernel size (3,3) and the last one is of size (2,2). And the number of filters is increased from 64 to 512 layer by layer. Two max-pooling layers are added with size (2,2) and then two max-pooling layers of size (2,1) are added to extract features with a larger width to predict long texts. Also, we used batch normalization layers after fifth and sixth convolution layers which accelerates the training process. Then we used a lambda function to squeeze the output from conv layer and make it compatible with LSTM layer. Then used two Bidirectional LSTM layers each of which has 128 units. This RNN layer gives the output of size (batch_size, 31, 63). Where 63 is the total number of output classes including blank character.

```
# input with shape of height=32 and width=128
```

```
inputs = Input(shape=(32,128,1))
```

```
# convolution layer with kernel size (3,3)
```

```
conv_1 = Conv2D(64, (3,3), activation = 'relu', padding='same')(inputs)
```

```
# poolig layer with kernel size (2,2)
```

```
pool_1 = MaxPool2D(pool_size=(2, 2), strides=2)(conv_1)
```

```
conv_2 = Conv2D(128, (3,3), activation = 'relu', padding='same')(pool_1)
```

```
pool_2 = MaxPool2D(pool_size=(2, 2), strides=2)(conv_2)
```

```
conv_3 = Conv2D(256, (3,3), activation = 'relu', padding='same')(pool_2)
```

```

conv_4 = Conv2D(256, (3,3), activation = 'relu', padding='same')(conv_3)

# poolig layer with kernel size (2,1)
pool_4 = MaxPool2D(pool_size=(2, 1))(conv_4)

conv_5 = Conv2D(512, (3,3), activation = 'relu', padding='same')(pool_4)

# Batch normalization layer
batch_norm_5 = BatchNormalization()(conv_5)

conv_6 = Conv2D(512, (3,3), activation = 'relu', padding='same')(batch_norm_5)
batch_norm_6 = BatchNormalization()(conv_6)
pool_6 = MaxPool2D(pool_size=(2, 1))(batch_norm_6)

conv_7 = Conv2D(512, (2,2), activation = 'relu')(pool_6)

squeezed = Lambda(lambda x: K.squeeze(x, 1))(conv_7)

# bidirectional LSTM layers with units=128
blstm_1 = Bidirectional(LSTM(256, return_sequences=True, dropout = 0.2))(squeezed)
blstm_2 = Bidirectional(LSTM(256, return_sequences=True, dropout = 0.2))(blstm_1)

outputs = Dense(len(char_list)+1, activation = 'softmax')(blstm_2)

# model to be used at test time
act_model = Model(inputs, outputs)

act_model.summary()

```



Model: "model_1"

Layer (type)	Output Shape	Param #
=====		
input_1 (InputLayer)	(None, 32, 128, 1)	0
conv2d_1 (Conv2D)	(None, 32, 128, 64)	640
max_pooling2d_1 (MaxPooling2)	(None, 16, 64, 64)	0
conv2d_2 (Conv2D)	(None, 16, 64, 128)	73856
max_pooling2d_2 (MaxPooling2)	(None, 8, 32, 128)	0
conv2d_3 (Conv2D)	(None, 8, 32, 256)	295168
conv2d_4 (Conv2D)	(None, 8, 32, 256)	590080
max_pooling2d_3 (MaxPooling2)	(None, 4, 32, 256)	0
conv2d_5 (Conv2D)	(None, 4, 32, 512)	1180160
batch_normalization_1 (Batch Normalization)	(None, 4, 32, 512)	2048
conv2d_6 (Conv2D)	(None, 4, 32, 512)	2359808
batch_normalization_2 (Batch Normalization)	(None, 4, 32, 512)	2048
max_pooling2d_4 (MaxPooling2)	(None, 2, 32, 512)	0
conv2d_7 (Conv2D)	(None, 1, 31, 512)	1049088
lambda_1 (Lambda)	(None, 31, 512)	0
bidirectional_1 (Bidirectional)	(None, 31, 512)	1574912
bidirectional_2 (Bidirectional)	(None, 31, 512)	1574912

see that I have only taken `y_pred` and neglected `y_true`. This is because I have already taken labels as input to the model earlier. labels as input to the model earlier.

Now train your model on 7850 training images and 876 validation images.

```
batch_size = 8
epochs = 30
e = str(epochs)
optimizer_name = 'sgd'

model.compile(loss={'ctc': lambda y_true, y_pred: y_pred}, optimizer = optimizer_name, met

filepath="gdrive/My Drive/TcsInternship/HTR_Using_CRNN/Model/{o-}{r-}{e-}{t-}{v}.hdf5".for
                                str(RECORDS_COUNT),
                                str(epochs),
                                str(train_images.shape[0]),
                                str(valid_images.shape[0]))

checkpoint = ModelCheckpoint(filepath=filepath, monitor='val_loss', verbose=1, save_best_o
callbacks_list = [checkpoint]

history = model.fit(x=[train_images, train_padded_label, train_input_length, train_label_l
                    y=np.zeros(len(train_images)),
                    batch_size=batch_size,
                    epochs=epochs,
                    validation_data=(valid_images, valid_padded_label, valid_input_length
                    verbose=2,
                    callbacks=callbacks_list)
```



Train on 7850 samples, validate on 876 samples

Epoch 1/30

- 229s - loss: 15.7982 - accuracy: 0.0000e+00 - val_loss: 22.7894 - val_accuracy: 0

Epoch 00001: val_loss improved from inf to 22.78938, saving model to gdrive/My Drive

Epoch 2/30

- 224s - loss: 13.4703 - accuracy: 0.0089 - val_loss: 12.8756 - val_accuracy: 0.026

Epoch 00002: val_loss improved from 22.78938 to 12.87564, saving model to gdrive/My

Epoch 3/30

- 222s - loss: 11.9471 - accuracy: 0.0466 - val_loss: 11.9610 - val_accuracy: 0.063

Epoch 00003: val_loss improved from 12.87564 to 11.96102, saving model to gdrive/My

Epoch 4/30

- 221s - loss: 10.4667 - accuracy: 0.0685 - val_loss: 16.7443 - val_accuracy: 0.067

Epoch 00004: val_loss did not improve from 11.96102

Epoch 5/30

- 217s - loss: 8.9951 - accuracy: 0.1096 - val_loss: 13.5724 - val_accuracy: 0.0890

Epoch 00005: val_loss did not improve from 11.96102

Epoch 6/30

- 216s - loss: 7.4734 - accuracy: 0.1517 - val_loss: 18.8284 - val_accuracy: 0.0205

Epoch 00006: val_loss did not improve from 11.96102

Epoch 7/30

- 216s - loss: 5.8994 - accuracy: 0.1939 - val_loss: 5.4707 - val_accuracy: 0.2295

Epoch 00007: val_loss improved from 11.96102 to 5.47070, saving model to gdrive/My D

Epoch 8/30

- 215s - loss: 4.7142 - accuracy: 0.2414 - val_loss: 19.6279 - val_accuracy: 0.0023

Epoch 00008: val_loss did not improve from 5.47070

Epoch 9/30

- 215s - loss: 3.8069 - accuracy: 0.2860 - val_loss: 4.2376 - val_accuracy: 0.3014

Epoch 00009: val_loss improved from 5.47070 to 4.23760, saving model to gdrive/My Dr

Epoch 10/30

- 216s - loss: 3.1251 - accuracy: 0.3362 - val_loss: 5.3671 - val_accuracy: 0.2386

Epoch 00010: val_loss did not improve from 4.23760

Epoch 11/30

- 216s - loss: 2.5277 - accuracy: 0.3862 - val_loss: 5.2800 - val_accuracy: 0.2820

Epoch 00011: val_loss did not improve from 4.23760

Epoch 12/30

- 214s - loss: 2.0851 - accuracy: 0.4413 - val_loss: 3.5682 - val_accuracy: 0.3721

Epoch 00012: val_loss improved from 4.23760 to 3.56821, saving model to gdrive/My Dr

Epoch 13/30

- 212s - loss: 1.6707 - accuracy: 0.4926 - val_loss: 3.4039 - val_accuracy: 0.4521

Epoch 00013: val_loss improved from 3.56821 to 3.40386, saving model to gdrive/My Dr

Epoch 14/30

- 212s - loss: 1.3409 - accuracy: 0.5468 - val_loss: 3.4710 - val_accuracy: 0.4532

Epoch 00014: val_loss did not improve from 3.40386

Epoch 15/30

- 213s - loss: 1.0556 - accuracy: 0.6051 - val_loss: 3.5048 - val_accuracy: 0.4772

Epoch 00015: val_loss did not improve from 3.40386

Epoch 16/30

- 214s - loss: 0.8307 - accuracy: 0.6648 - val_loss: 3.5462 - val_accuracy: 0.5068

Epoch 00016: val_loss did not improve from 3.40386

Epoch 17/30

- 214s - loss: 0.6426 - accuracy: 0.7224 - val_loss: 3.5457 - val_accuracy: 0.5160

Epoch 00017: val_loss did not improve from 3.40386

Epoch 18/30

- 219s - loss: 0.4928 - accuracy: 0.7732 - val_loss: 4.1715 - val_accuracy: 0.4212

Epoch 00018: val_loss did not improve from 3.40386

Epoch 19/30

- 218s - loss: 0.4007 - accuracy: 0.8079 - val_loss: 3.3514 - val_accuracy: 0.5354

Epoch 00019: val_loss improved from 3.40386 to 3.35142, saving model to gdrive/My Dr

Epoch 20/30

- 221s - loss: 0.2980 - accuracy: 0.8544 - val_loss: 12.1324 - val_accuracy: 0.1450

Epoch 00020: val_loss did not improve from 3.35142

Epoch 21/30

- 221s - loss: 0.2732 - accuracy: 0.8730 - val_loss: 3.5094 - val_accuracy: 0.5468

Epoch 00021: val_loss did not improve from 3.35142

Epoch 22/30

- 222s - loss: 0.2030 - accuracy: 0.9066 - val_loss: 3.6616 - val_accuracy: 0.5776

Epoch 00022: val_loss did not improve from 3.35142

Epoch 23/30

- 223s - loss: 0.1655 - accuracy: 0.9237 - val_loss: 3.6411 - val_accuracy: 0.5685

Epoch 00023: val_loss did not improve from 3.35142

Epoch 24/30

- 224s - loss: 0.1268 - accuracy: 0.9434 - val_loss: 3.6614 - val_accuracy: 0.5833

Epoch 29/30

Requirement already satisfied: setuptools in /usr/local/lib/python3.6/dist-packages
Building wheels for collected packages: python-levenshteinBuilding wheel for python-levenshtein (setup.py) ... done

Created wheel for python-levenshtein: filename=python-Levenshtein-0.12.0-cp36-cp36 Stored in directory: /root/.cache/pip/wheels/de/c2/93/660fd5f7559049268ad2dc6d81c4

Successfully built python-levenshtein

Installing collected packages: python-levenshtein

Successfully installed python-levenshtein-0.12.0

Prediction.

```
# predict outputs on validation images
```

```
prediction =act_model.predict(train_images[542:645])
```

```
# use CTC decoder
```

```
decoded = K.ctc_decode(prediction,  
                        input_length=np.ones(prediction.shape[0]) * prediction.shape[1],  
                        greedy=True)[0][0]
```

```
out = K.get_value(decoded)
```

```
# see the results
```

```
for i, x in enumerate(out):
    print("original_text = ", train_original_text[542+i])
    print("predicted text = ", end = '')
    for p in x:
        if int(p) != -1:
            print(char_list[int(p)], end = '')
    plt.imshow(train_images[542+i].reshape(32,128), cmap=plt.cm.gray)
    plt.show()
    print('\n')
```



CONCLUSION

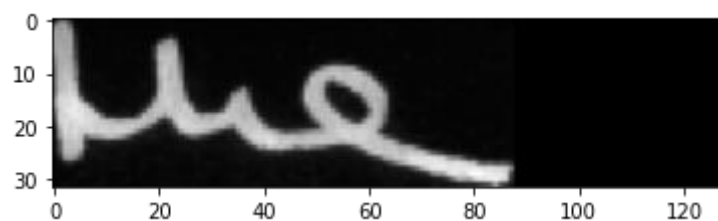
In this project, we successfully developed a deep learning-based system to automate the text recognition and transliteration of historical documents using a weighted convolutional recurrent architecture. The model integrates the power of Convolutional Neural Networks (CNNs) for feature extraction and Bidirectional LSTMs (BiLSTMs) for sequence modeling, paired with CTC loss to handle unsegmented sequence labeling. This approach enables robust recognition of handwritten or printed historical scripts, even with degradation or stylistic variance.

To extend usability, we incorporated a transliteration component that converts recognized characters from the source script (e.g., Devanagari) into a target script (e.g., Latin), aiding researchers, linguists, and archivists in interpreting and digitizing ancient manuscripts. The system lays the groundwork for more advanced applications in historical text analysis, including:

- Multi-lingual and multi-script document parsing,
- Integration with language models for contextual correction,
- Real-time digitization pipelines for archives and museums

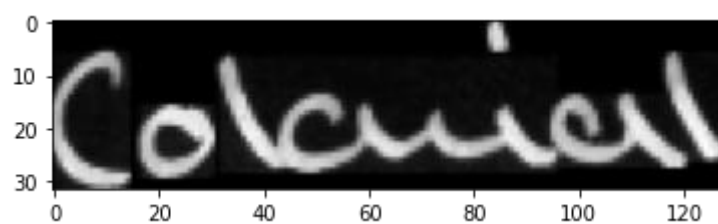
original_text = the

predicted text = the



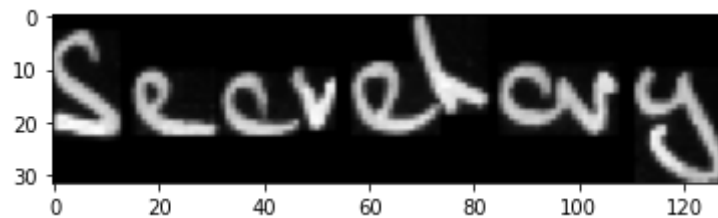
original_text = Colonial

predicted text = Colonial



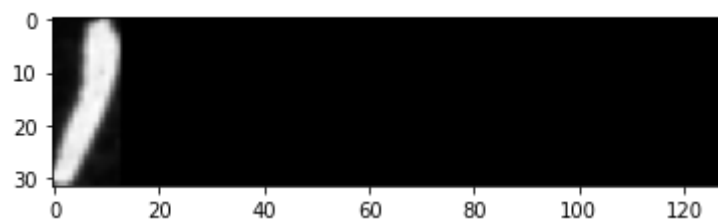
original_text = Secretary

predicted text = Secretary



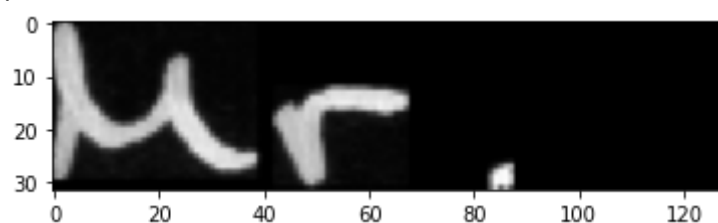
original_text = ,

predicted text = ,

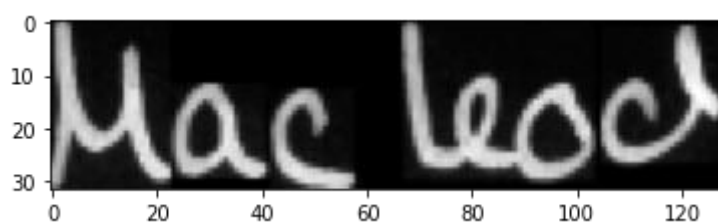


original_text = Mr.

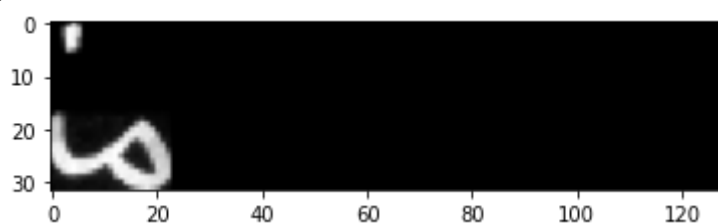
predicted text = Mr.



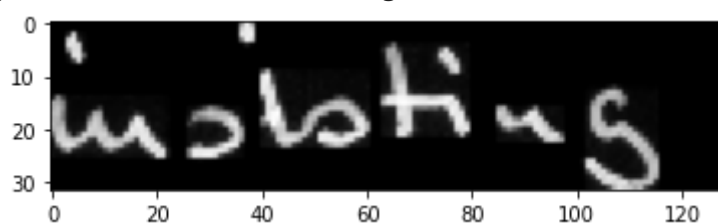
original_text = Macleod
predicted text = Macleod



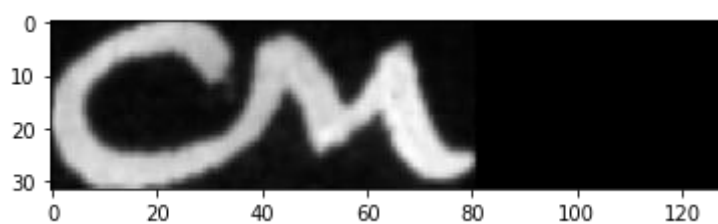
original_text = is
predicted text = is



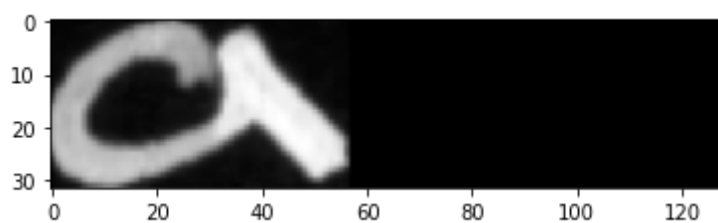
original_text = insisting
predicted text = insisting



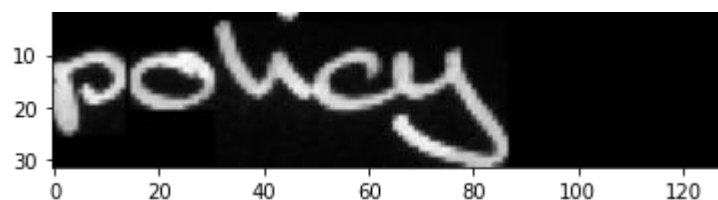
original_text = on
predicted text = on



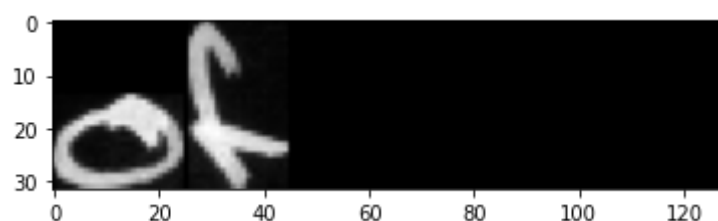
original_text = a
predicted text = a



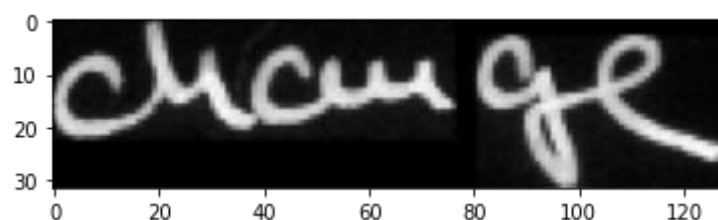
```
original_text = policy  
predicted text = policy
```



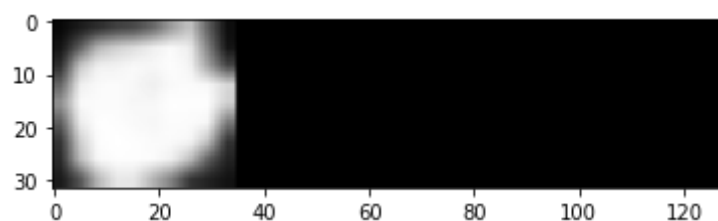
```
original_text = of  
predicted text = of
```



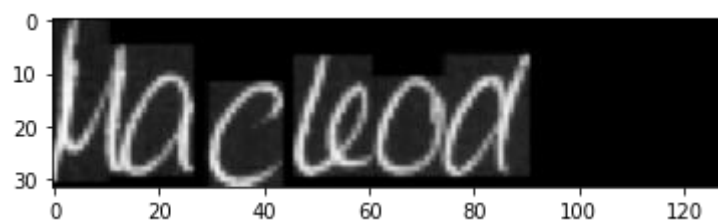
```
original_text = change  
predicted text = change
```



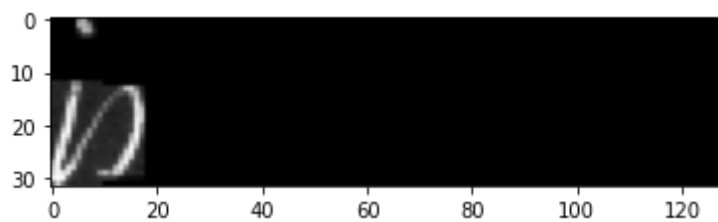
```
original_text = .  
predicted text = .
```



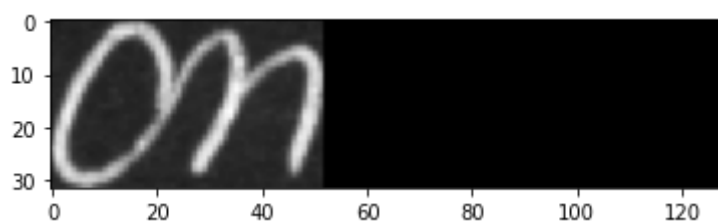
```
original_text = Macleod  
predicted text = Macleod
```



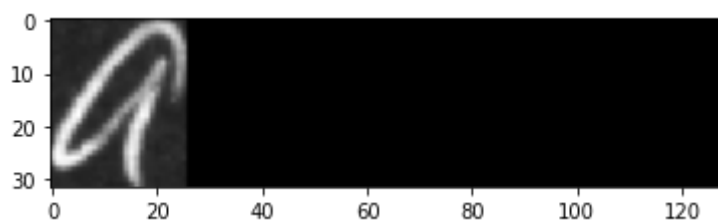
original_text = is
predicted text = is



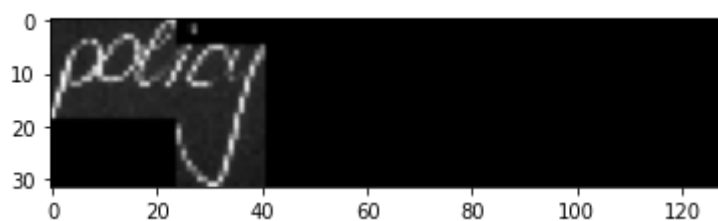
original_text = on
predicted text = on



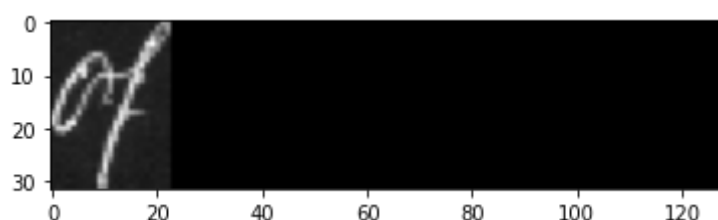
original_text = a
predicted text = a



original_text = policy
predicted text = policy



original_text = of
predicted text = of

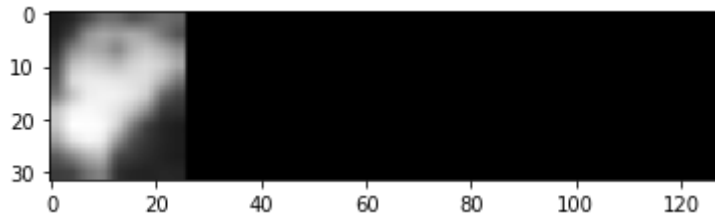


original_text = change
predicted text = change

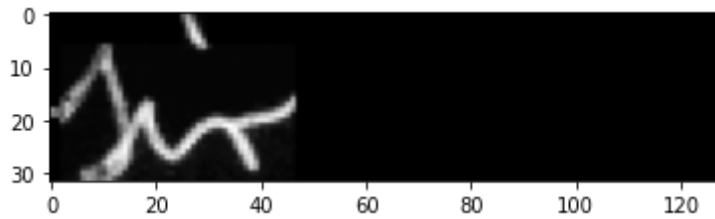




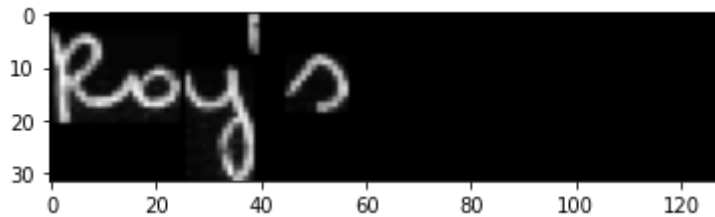
original_text = .
predicted text = .



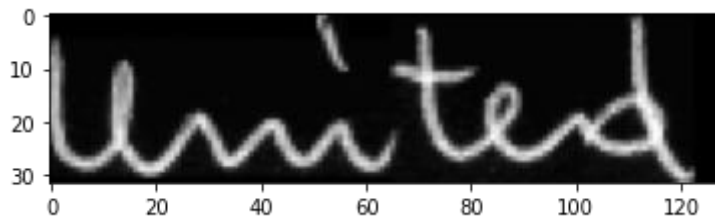
original_text = Sir
predicted text = Sir



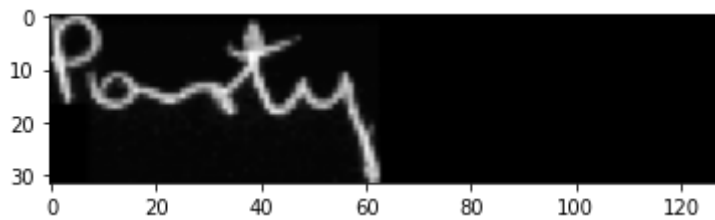
original_text = Roy's
predicted text = Roy's



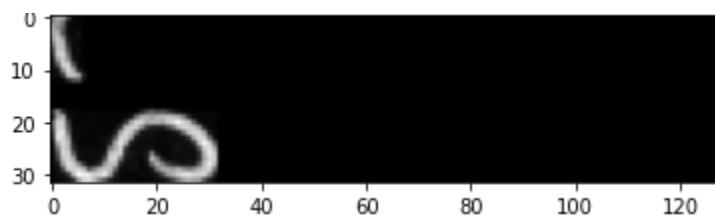
original_text = United
predicted text = United



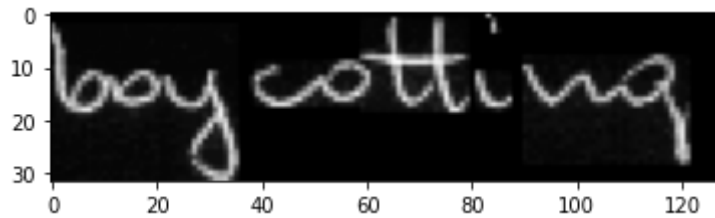
original_text = Party
predicted text = Party



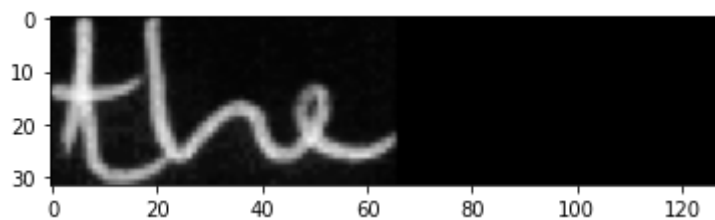
original_text = is
predicted text = is



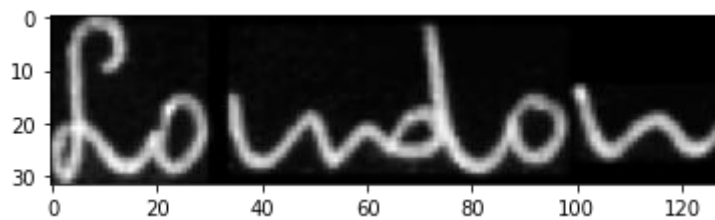
original_text = boycotting
predicted text = boycotting



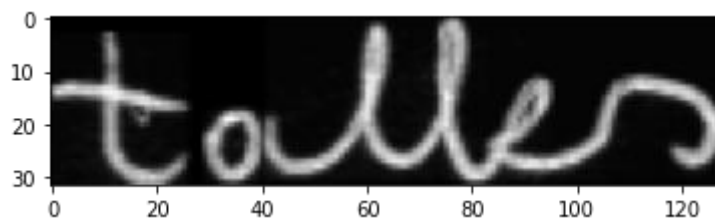
original_text = the
predicted text = the



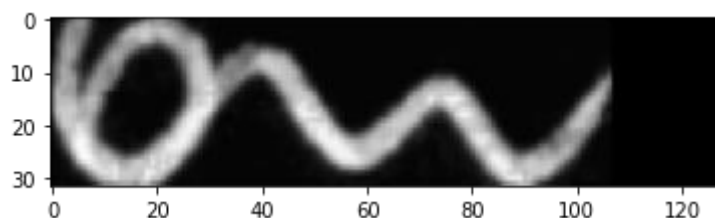
original_text = London
predicted text = London



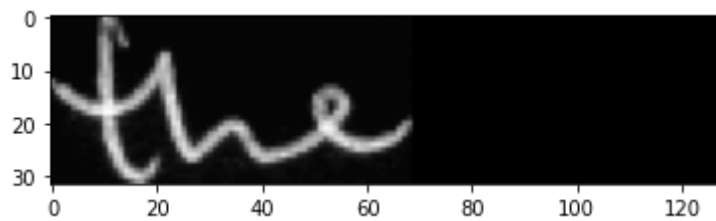
original_text = talks
predicted text = talks



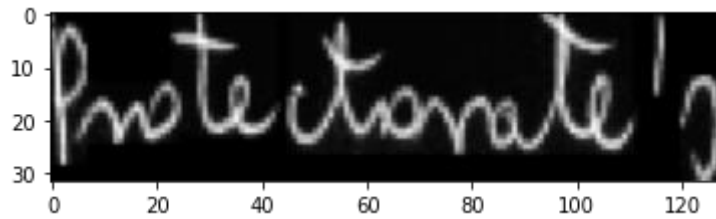
original_text = on
predicted text = on



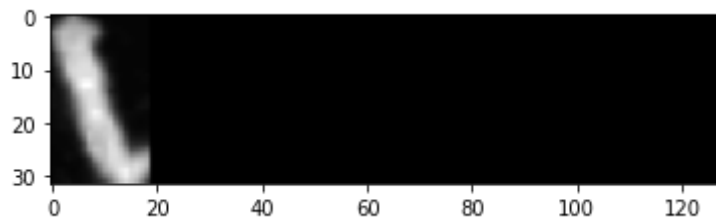
original_text = the
predicted text = the



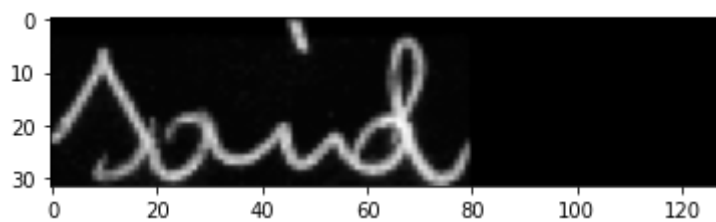
original_text = Protectorate's
predicted text = Protectorate's



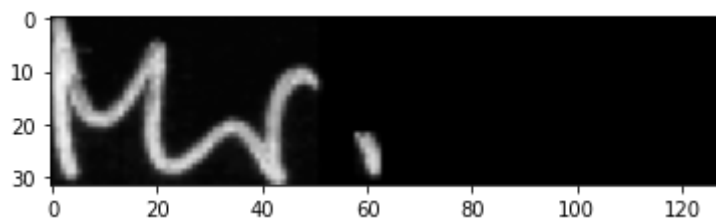
original_text = .
predicted text = .



original_text = Said
predicted text = Said

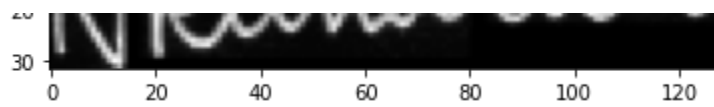


original_text = Mr.
predicted text = Mr.

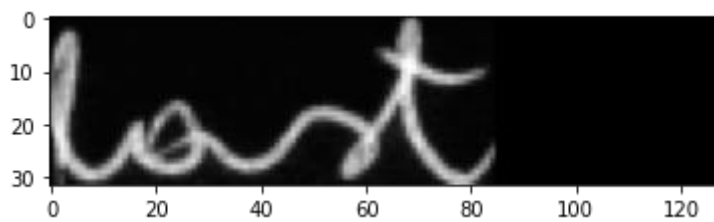


original_text = Nkumbula
predicted text = Nkumbula

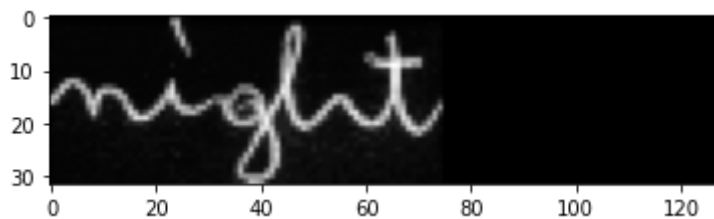




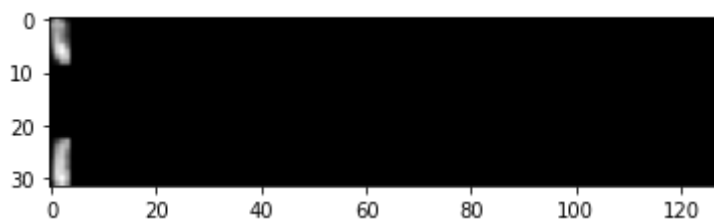
original_text = last
predicted text = last



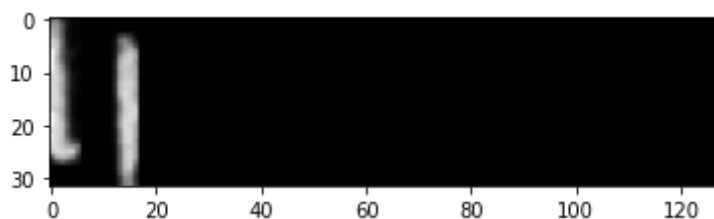
original_text = night
predicted text = night



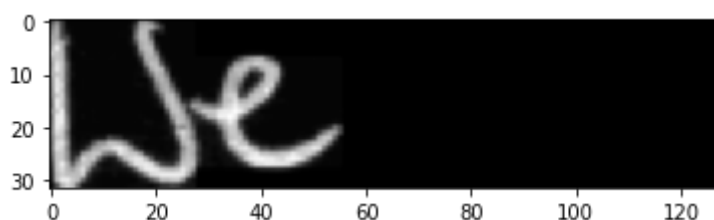
original_text = :
predicted text = :



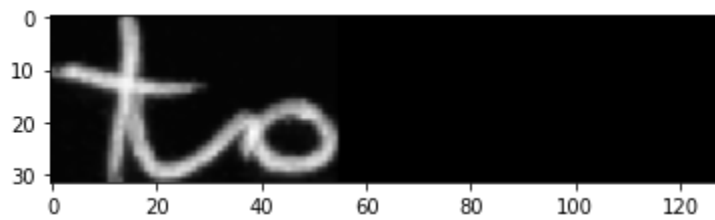
original_text = "
predicted text = "



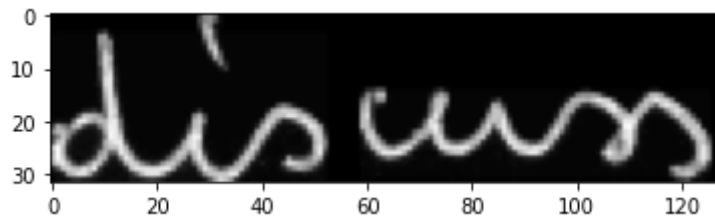
original_text = We
predicted text = We



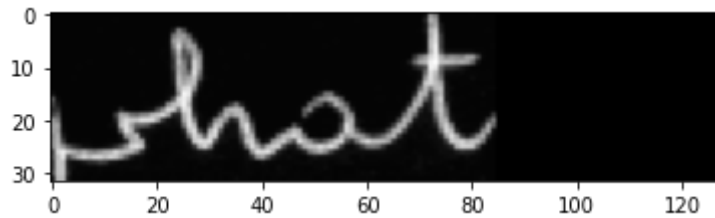
original_text = to
predicted text = to



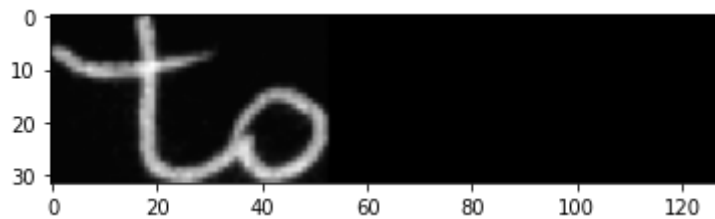
original_text = discuss
predicted text = discuss



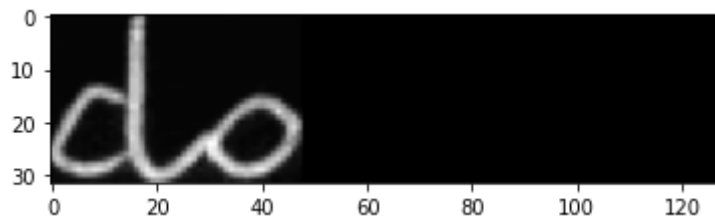
original_text = what
predicted text = what



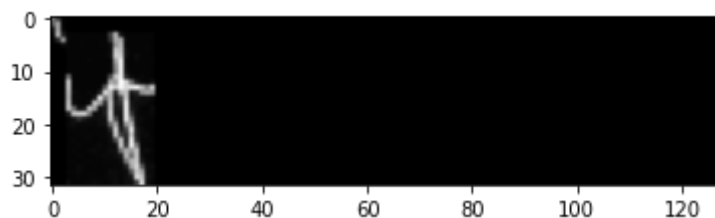
original_text = to
predicted text = to



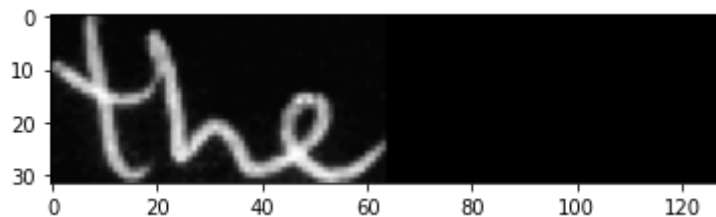
original_text = do
predicted text = do



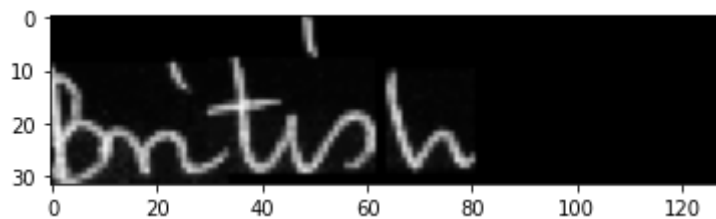
original_text = if
predicted text = if



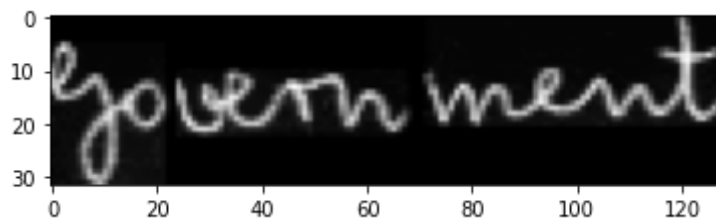
original_text = the
predicted text = the



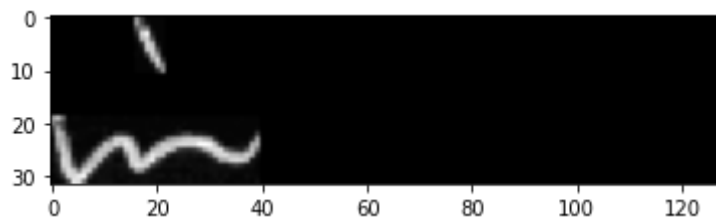
original_text = British
predicted text = British



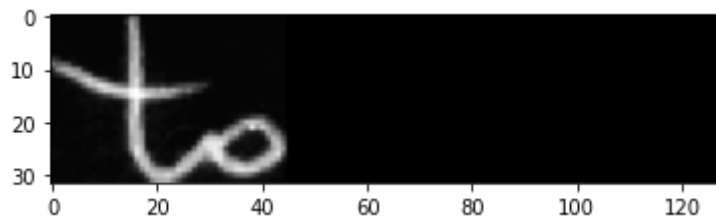
original_text = Government
predicted text = Government



original_text = in
predicted text = in

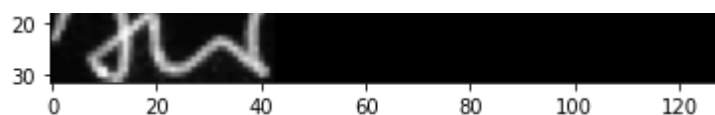


original_text = to
predicted text = to

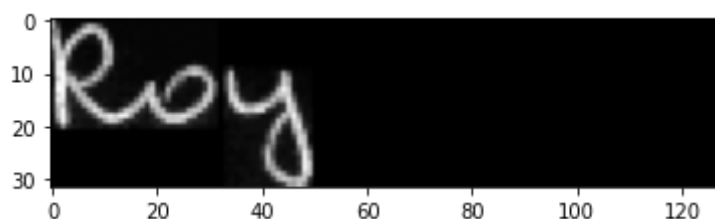


original_text = Sir
predicted text = Sir

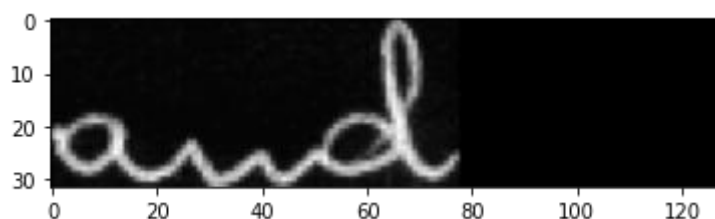




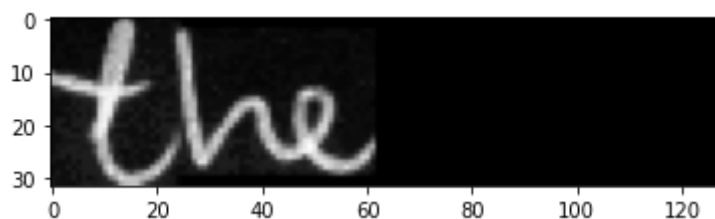
original_text = Roy
predicted text = Roy



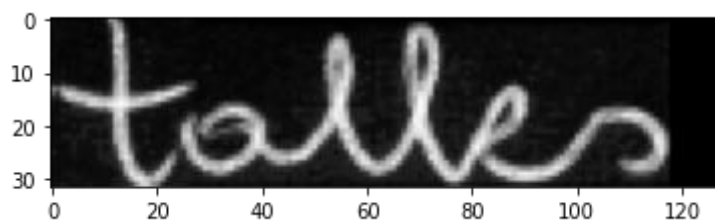
original_text = and
predicted text = and



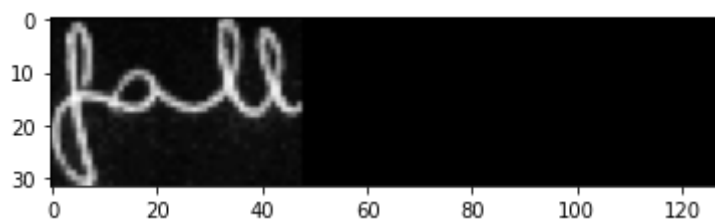
original_text = the
predicted text = the



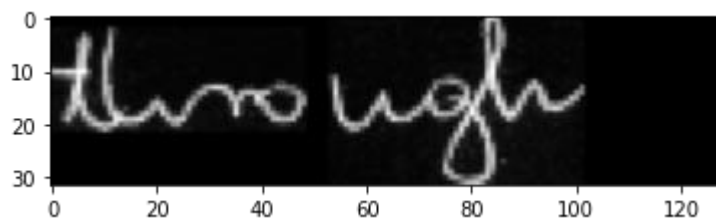
original_text = talks
predicted text = talks



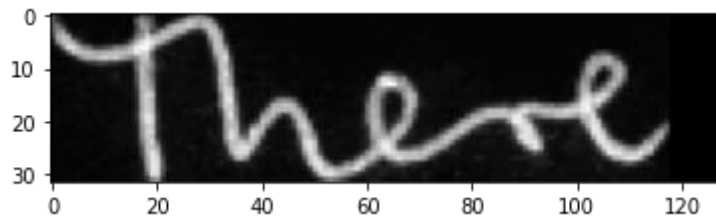
original_text = fall
predicted text = fall



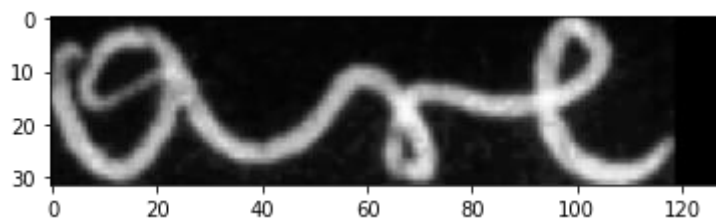
original_text = through
predicted text = through



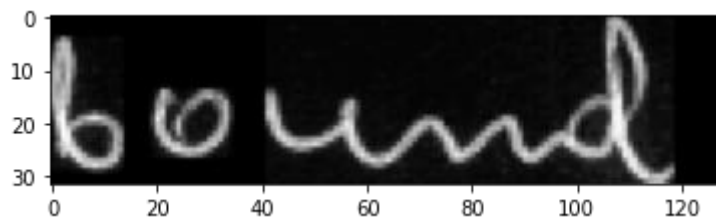
original_text = There
predicted text = There



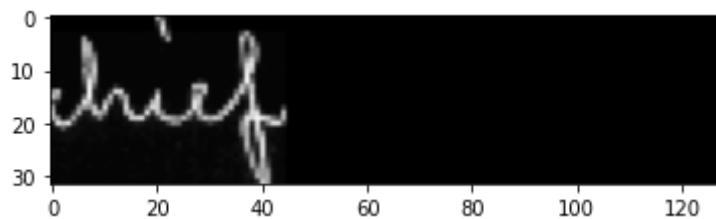
original_text = are
predicted text = are



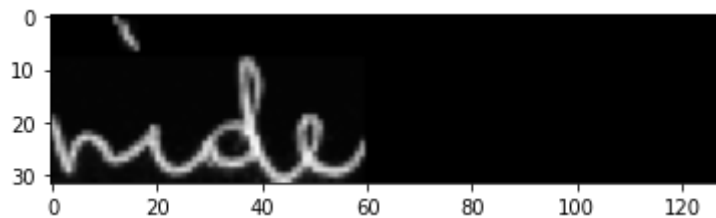
original_text = bound
predicted text = bound



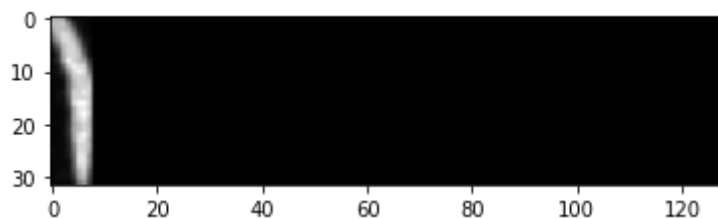
original_text = chief
predicted text = chief



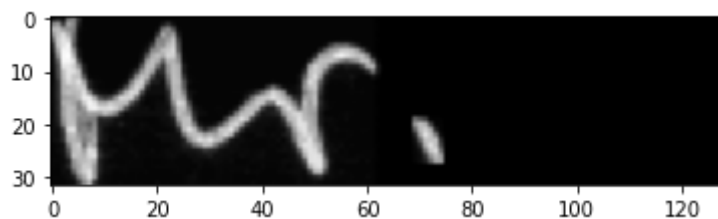
original_text = aide
predicted text = aide



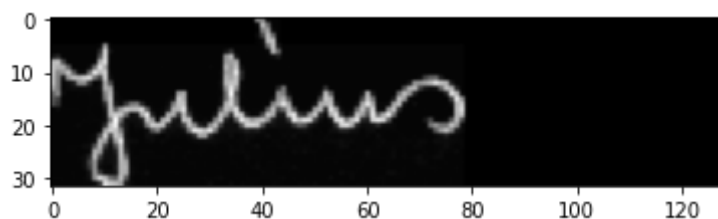
original_text = ,
predicted text = ,



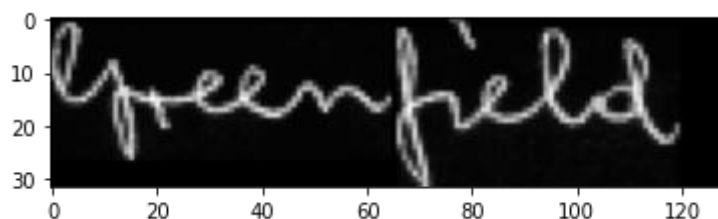
original_text = Mr.
predicted text = Mr.



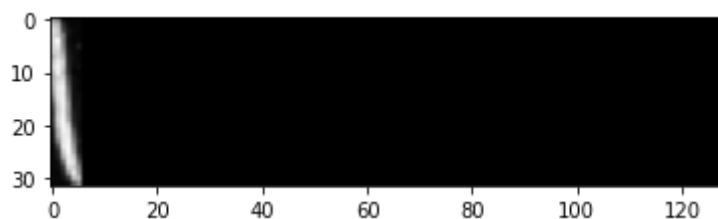
original_text = Julius
predicted text = Julius



original_text = Greenfield
predicted text = Geenfield

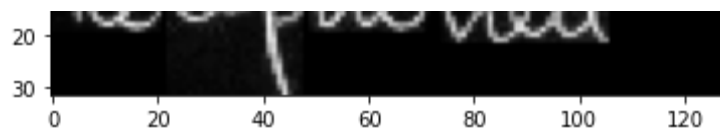


original_text = ,
predicted text = ,

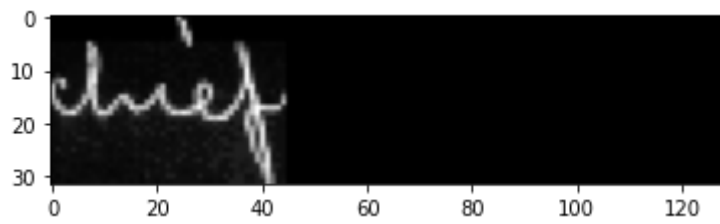


original_text = telephoned
predicted text = telephoned

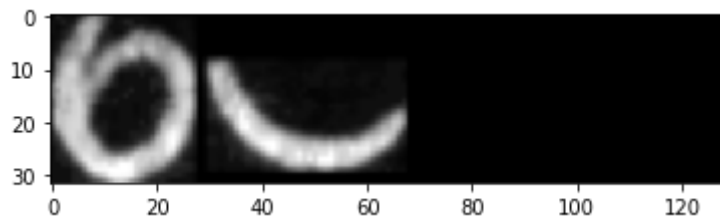




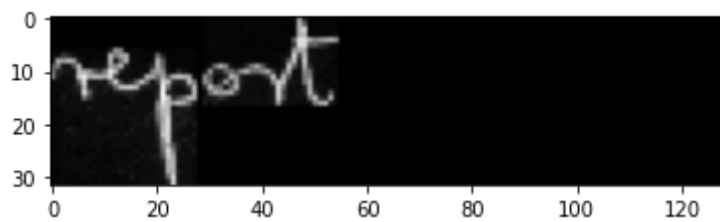
original_text = chief
predicted text = chief



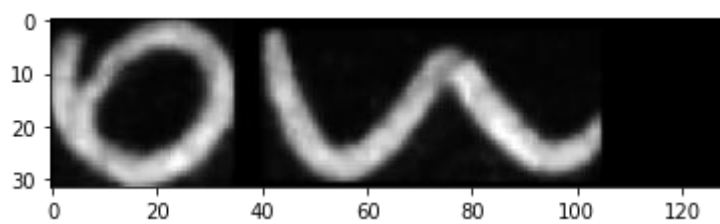
original_text = a
predicted text = a



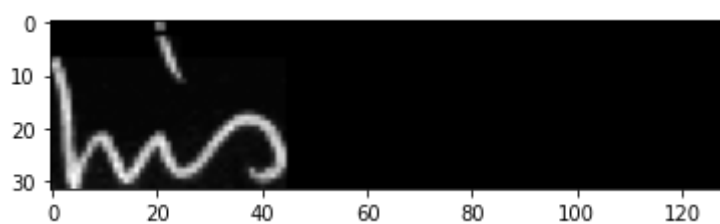
original_text = report
predicted text = report



original_text = on
predicted text = on

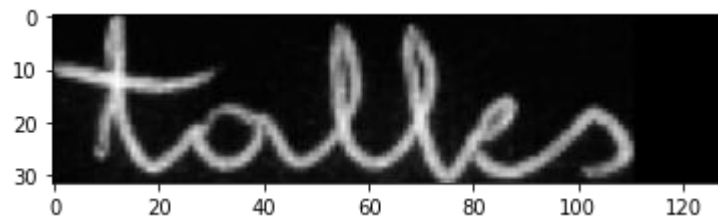


original_text = his
predicted text = his



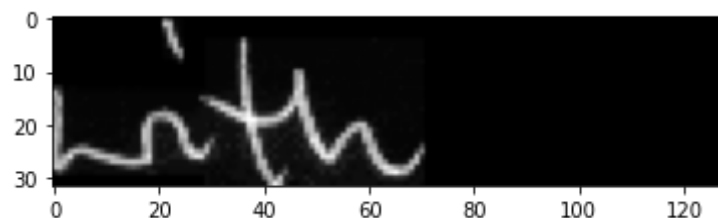
original_text = talks

predicted text = tals



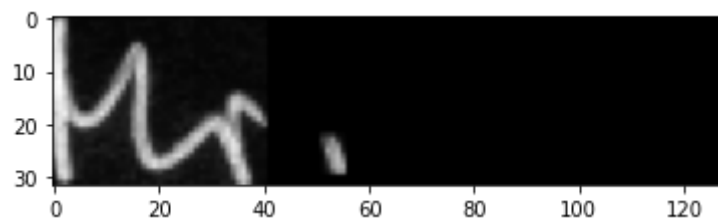
original_text = with

predicted text = with



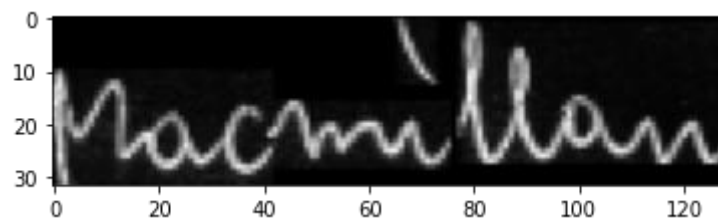
original_text = Mr.

predicted text = Mr.



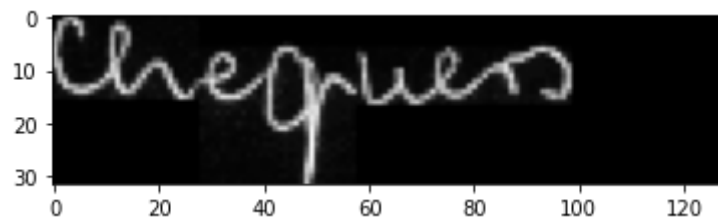
original_text = Macmillan

predicted text = Macmillan



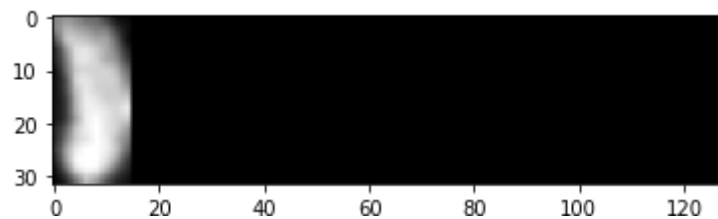
original_text = Chequers

predicted text = Chequers

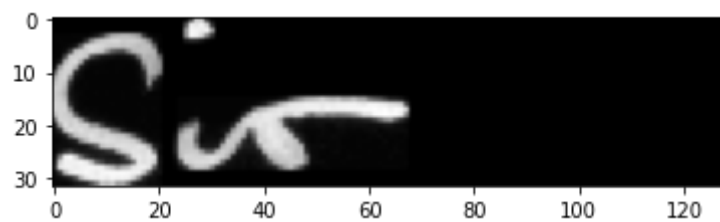


original_text = .

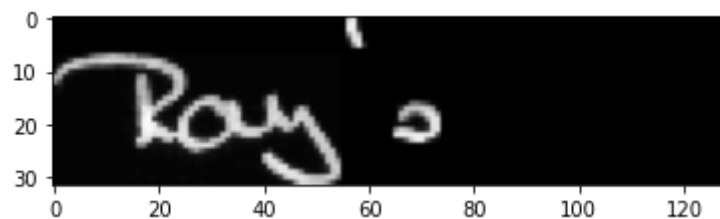
predicted text = .



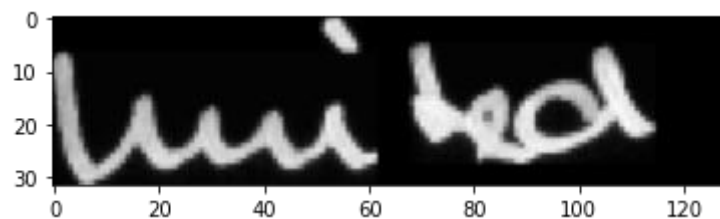
original_text = Sir
predicted text = Sir



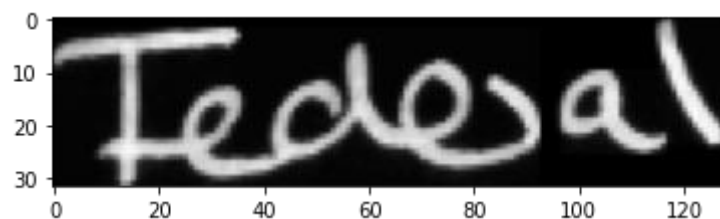
original_text = Roy's
predicted text = Roy's



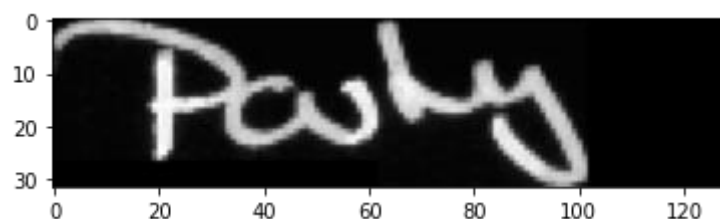
original_text = United
predicted text = United



original_text = Federal
predicted text = Federal

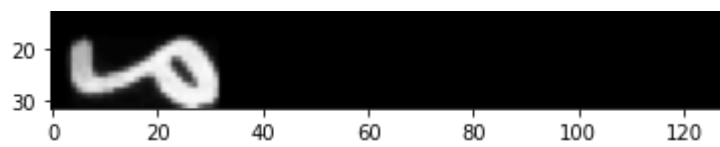


original_text = Party
predicted text = Party

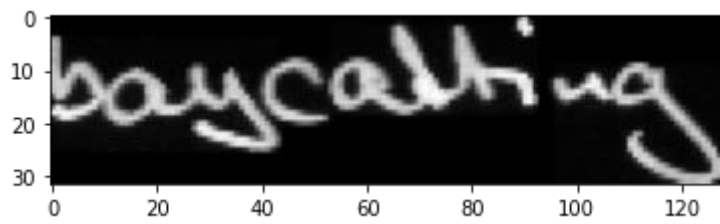


original_text = is
predicted text = is

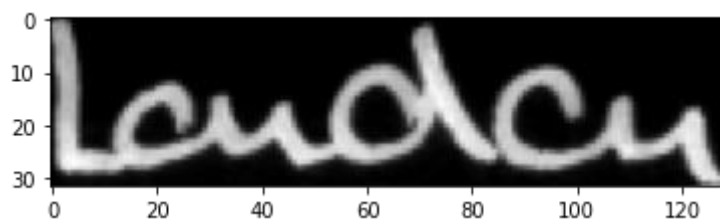




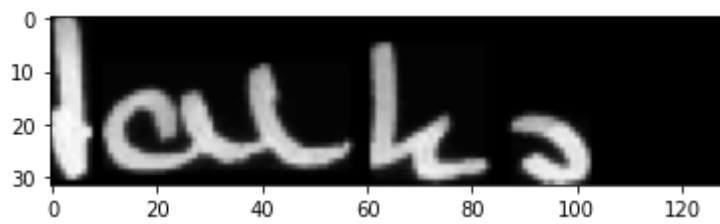
original_text = boycotting
predicted text = boycotting



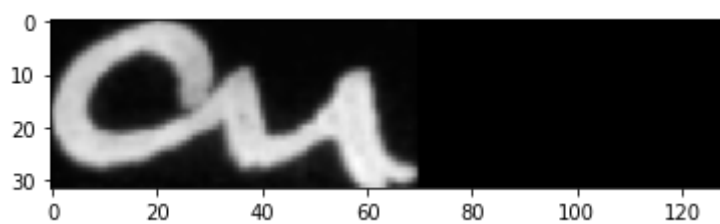
original_text = London
predicted text = London



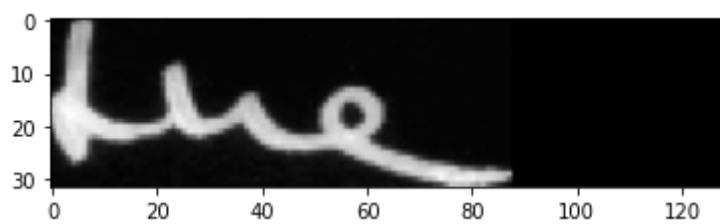
original_text = talks
predicted text = talks



original_text = on
predicted text = on

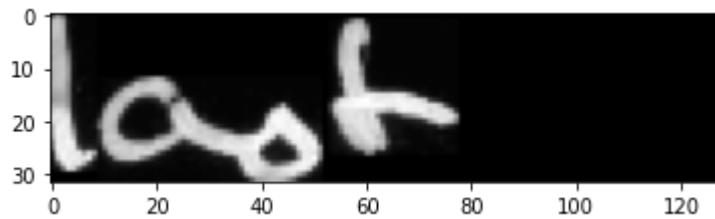


original_text = the
predicted text = the

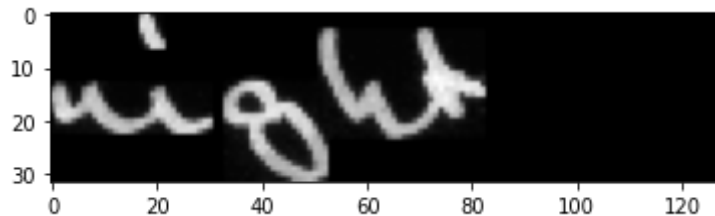


original_text = last

```
predicted text = last
```



```
original_text = night  
predicted text = night
```



```
original_text = :  
predicted text = :
```

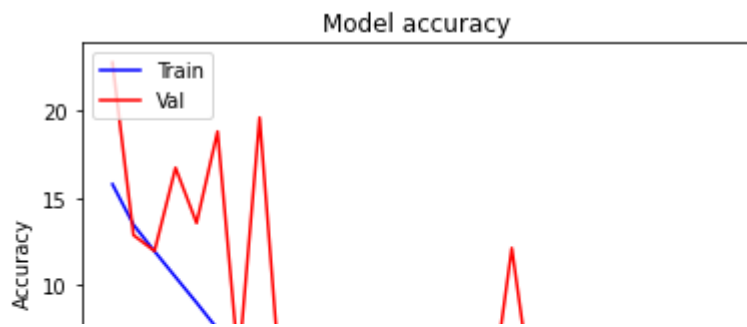


▼ Plot Accuracy and Loss

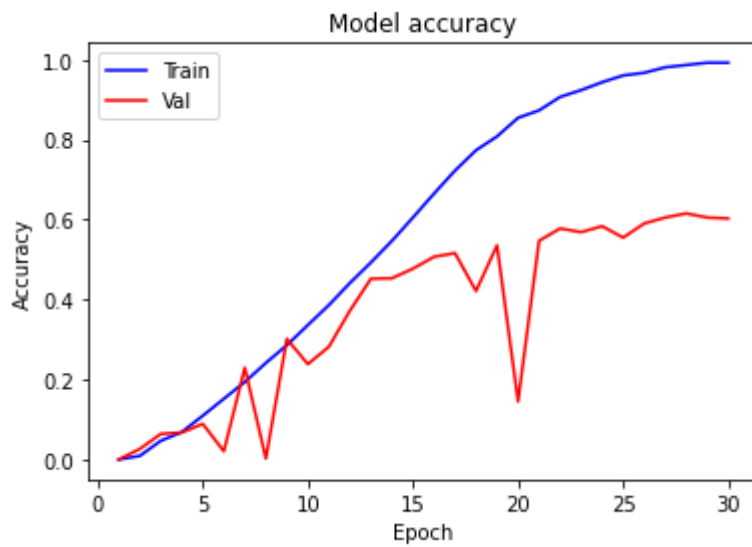
```
def plotgraph(epochs, acc, val_acc):  
    # Plot training & validation accuracy values  
    plt.plot(epochs, acc, 'b')  
    plt.plot(epochs, val_acc, 'r')  
    plt.title('Model accuracy')  
    plt.ylabel('Accuracy')  
    plt.xlabel('Epoch')  
    plt.legend(['Train', 'Val'], loc='upper left')  
    plt.show()
```

```
acc = history.history['accuracy']  
val_acc = history.history['val_accuracy']  
loss = history.history['loss']  
val_loss = history.history['val_loss']  
epochs = range(1, len(loss) + 1)  
  
0      20     40     60     80     100    120  
  
plotgraph(epochs, loss, val_loss)
```





```
plotgraph(epochs, acc, val_acc)
```



▼ Get Best Model Index

```
minimum_val_loss = np.min(history.history['val_loss'])
best_model_index = np.where(history.history['val_loss'] == minimum_val_loss)[0][0]

best_loss = str(history.history['loss'][best_model_index])
best_acc = str(history.history['accuracy'][best_model_index])
best_val_loss = str(history.history['val_loss'][best_model_index])
best_val_acc = str(history.history['val_accuracy'][best_model_index])
```